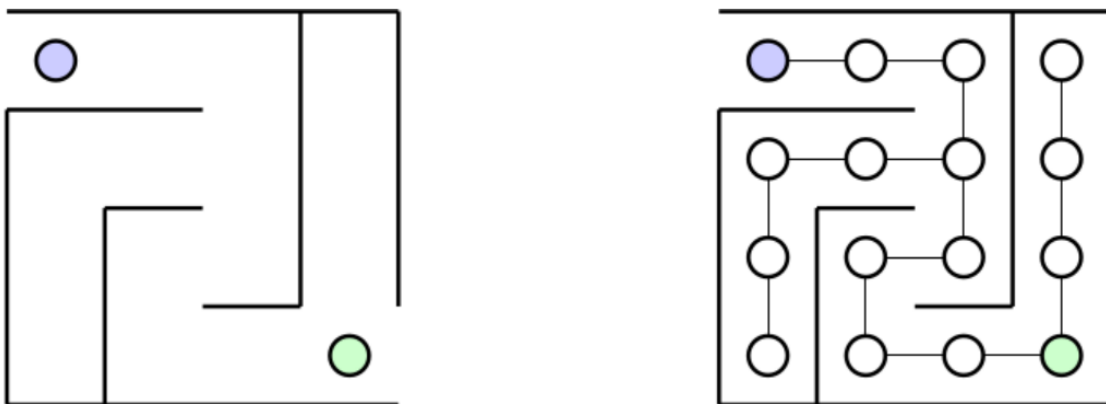


Nom	Prénom

I. Parcours d'un Labyrinthe

Vous vous trouvez dans un labyrinthe contenant de nombreux chemins possibles, et vous cherchez la sortie alors que vous ne connaissez aucunes indications sur ce labyrinthe. Tout d'abord, on peut représenter notre labyrinthe comme un graphe dit implicite, qui sera non pondéré (chaque arc aura alors une distance de 1 unité).

En effet, chaque intersection sera représentée par un nœud du graphe, et chaque chemin par un arc. L'entrée et la sortie du labyrinthe sont juste de simples nœuds distincts du graphe, et le fait de trouver la sortie du labyrinthe, revient à trouver un chemin reliant le nœud d'entrée au nœud de sortie :



Exemple de représentation d'un labyrinthe sous forme de graphe

Vu qu'on ne connaît rien sur ce labyrinthe, il est impossible de deviner le chemin nous permettant d'arriver à la sortie puisque ça pourrait être n'importe lequel du graphe. On peut donc tout simplement essayer chacun des chemins jusqu'à trouver le bon. Ce parcours est alors qualifié de : parcours en profondeur.

Principe

Le parcours en profondeur (ou plus communément DFS pour Depth First Search) permet de parcourir un graphe en utilisant le principe de la récursivité. Ce parcours visite les nœuds du graphe les plus "profonds" en premier (c'est-à-dire les plus éloignés du nœud de départ), avant de "remonter" progressivement dans le graphe.

Dans notre cas du labyrinthe, on essaie un chemin jusqu'à être bloqué, puis on revient à la dernière intersection, on continue jusqu'à être bloqué, on revient à la dernière intersection, etc. jusqu'à tomber sur la sortie.

L'objectif de ce TP est de découvrir comment les notions de parcours en profondeur et en largeur peuvent être utilisées dans le cas des labyrinthes.

Un labyrinthe sera représenté de la façon suivante : c'est une matrice d'entiers, qui valent 0 sur les zones dégagées et 1 sur les murs. On suppose que tous les coefficients du bord de la matrice sont des 1. Une seule case de la matrice contient l'entier 2, c'est la sortie. On a le droit de se déplacer dans les quatre directions, mais pas en diagonale.

Exemple :

```
[1;1;1;1;1;1;1;1;1];
```

```
[1;0;1;0;0;1;2;1;1];
```

```
[1;0;1;0;1;0;0;0;1];
```

```
[1;0;0;0;0;0;1;0;1];
```

```
[1;1;1;1;1;1;1;1;1]
```

```
[];
```

- Expliquez de manière très claire la notion de graphe (connexe biparti etc...)
- Faites une représentation du labyrinthe sous forme de graphe.
- Ecrire un programme (C, C++, Python, Java,) qui permet de parcourir le labyrinthe. Le programme doit montrer le trajet (en couleur) Par exemple on lance un petit lutin dans le labyrinthe à la case $i_0; j_0$ et on le charge de trouver la sortie. On lui donne un petit pot de

peinture, comme ça il peut marquer les cases déjà rencontrées. Naturellement notre lutin va effectuer un parcours en profondeur.

d. On voudrait maintenant connaître le plus court chemin de l'entrée à la sortie.

Pour cela, au lieu de s'enfoncer le plus possible dans les chemins comme avec un parcours en profondeur, on va plutôt regarder d'abord les cases qui sont à distance 1, puis celles qui sont à distance 2, etc. C'est l'idée du parcours en largeur. Pour cela, on va utiliser une file d'attente (structure FIFO) pour stocker les cases à explorer (et on a donc besoin d'un lutin plus intelligent).